LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Denny Mulia 2509106011

Kelas A1‘25

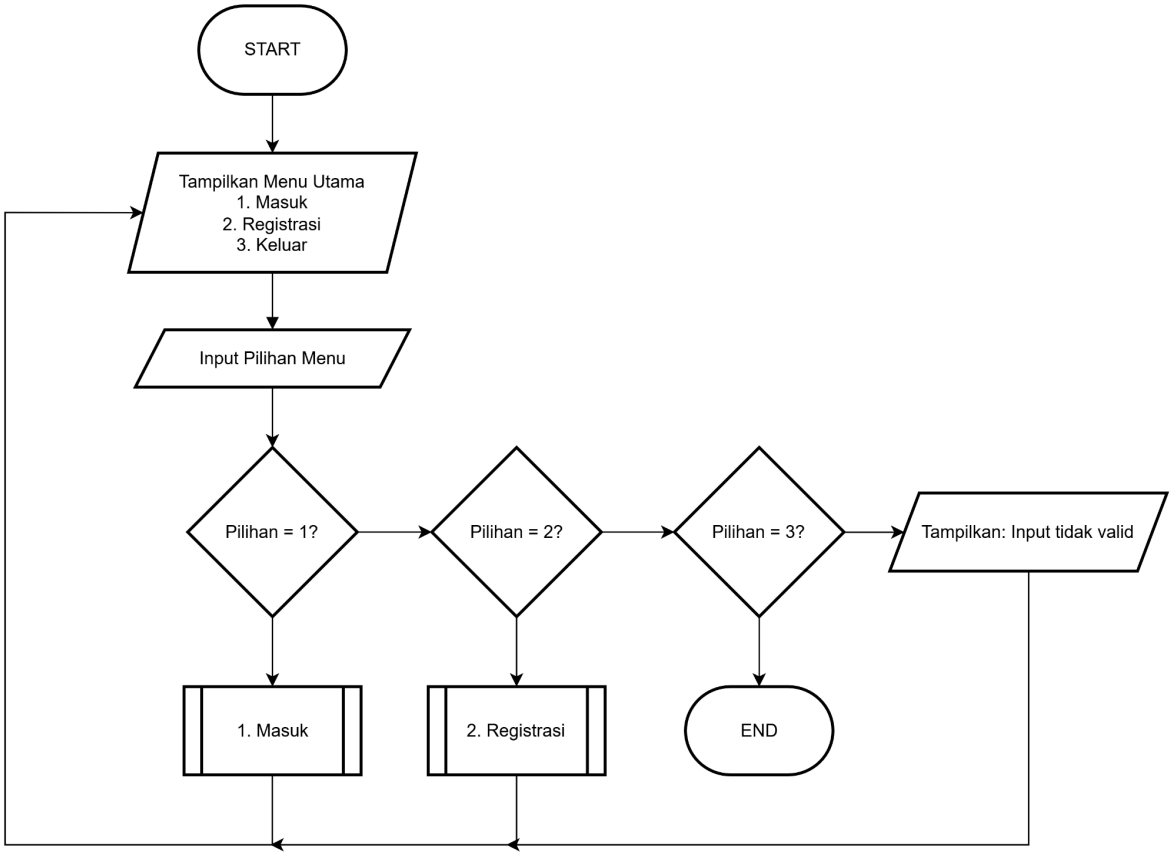
PROGRAM STUDI INFORMATIKA

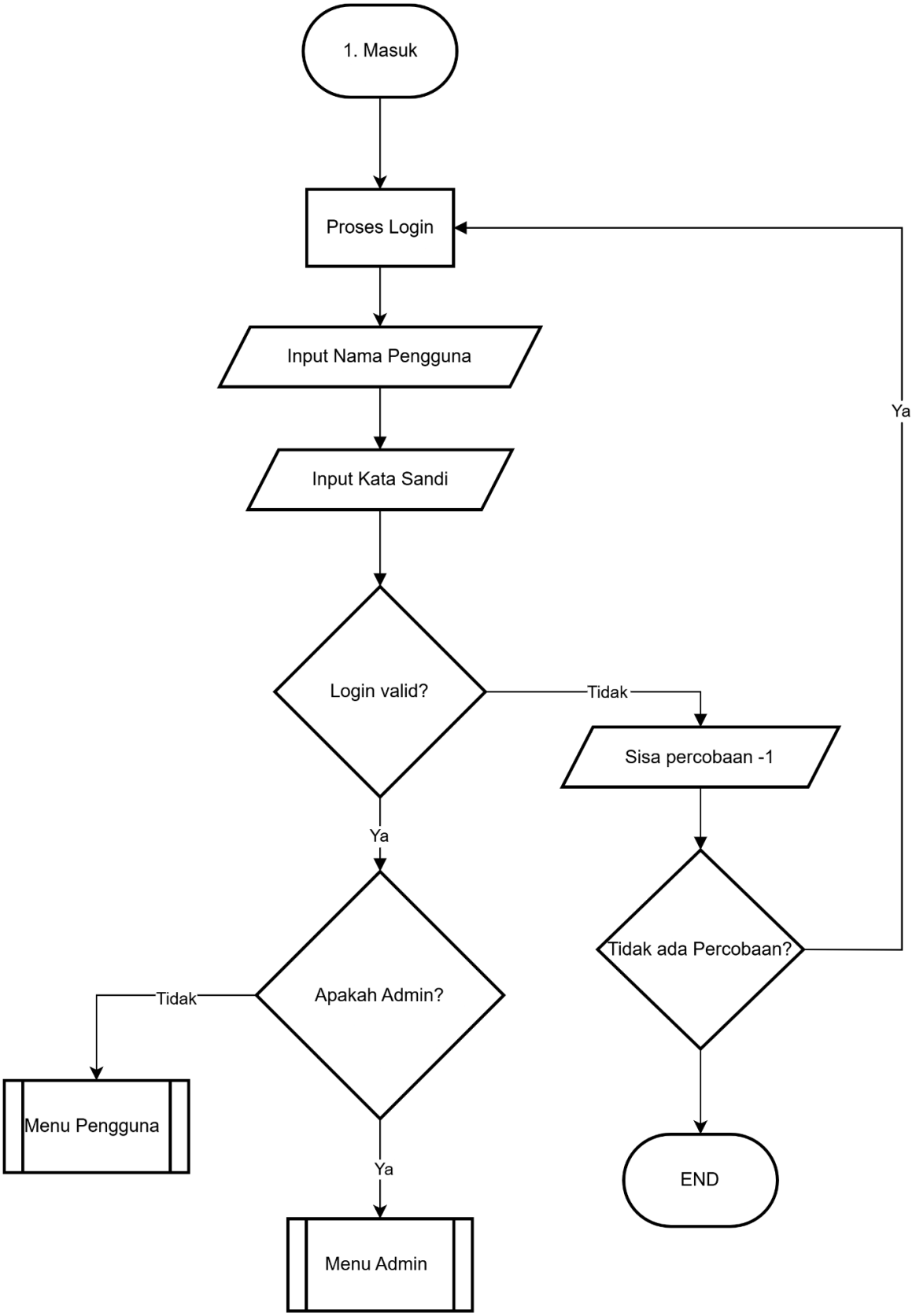
UNIVERSITAS MULAWARMAN

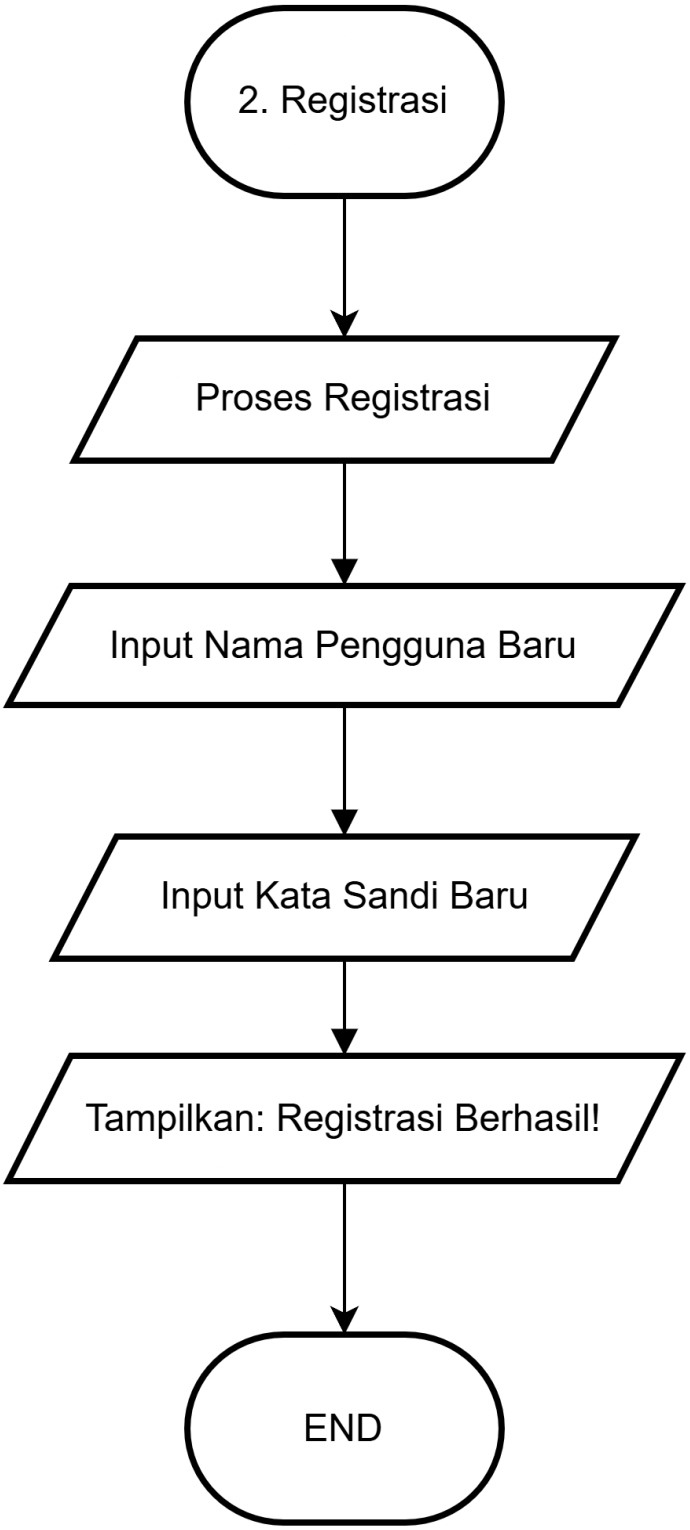
SAMARINDA

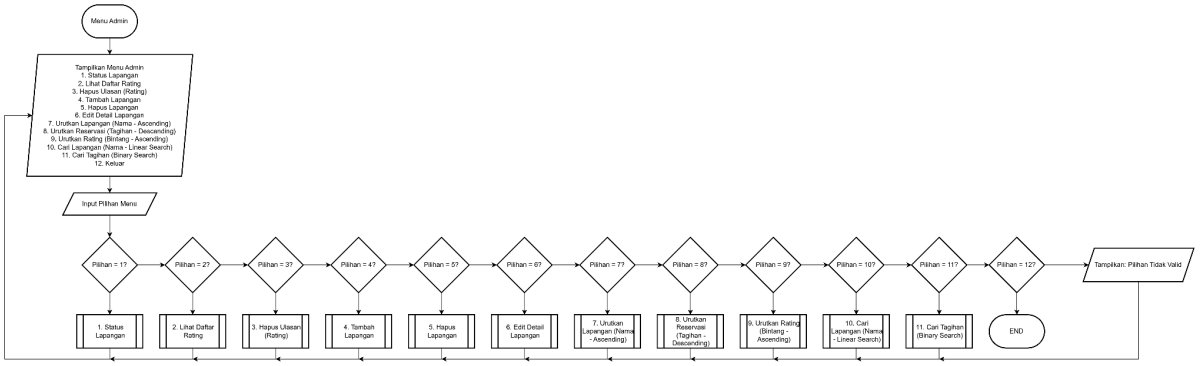
2025

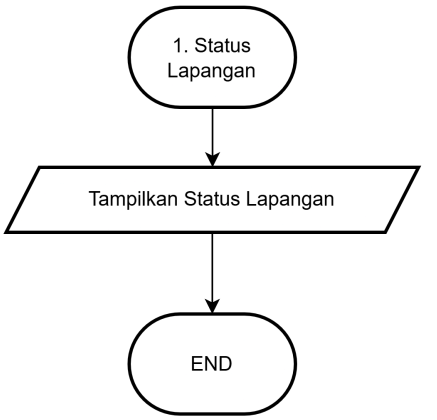
1. Flowchart

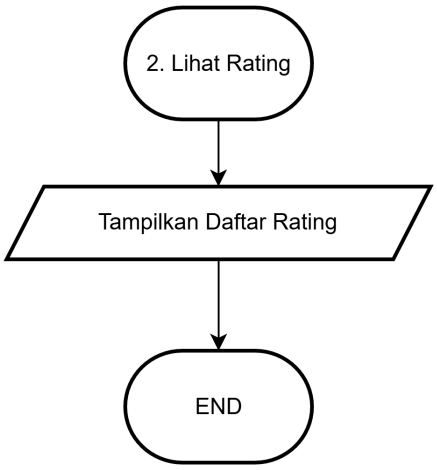

START
Tampilkan Menu Utama
1. Masuk
2. Registrasi
3. Keluar
Input Pilihan Menu
Pilihan = 1?
Pilihan = 2?
Pilihan = 3?
Tampilkan: Input tidak valid
1. Masuk
2. Registrasi
END

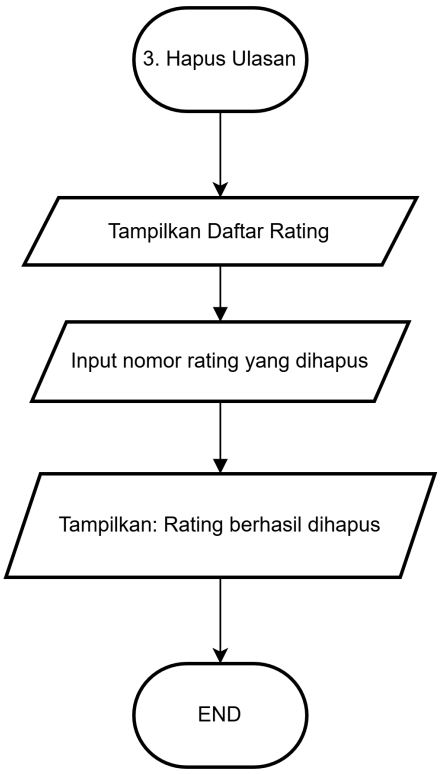

1. Masuk
Proses Login
Input Nama Pengguna
Input Kata Sandi
Login valid?
Tidak
Sisa percobaan -1
Ya
Tidak ada Percobaan?
Ya
Apakah Admin?
Tidak
Menu Pengguna
Ya
Menu Admin
END

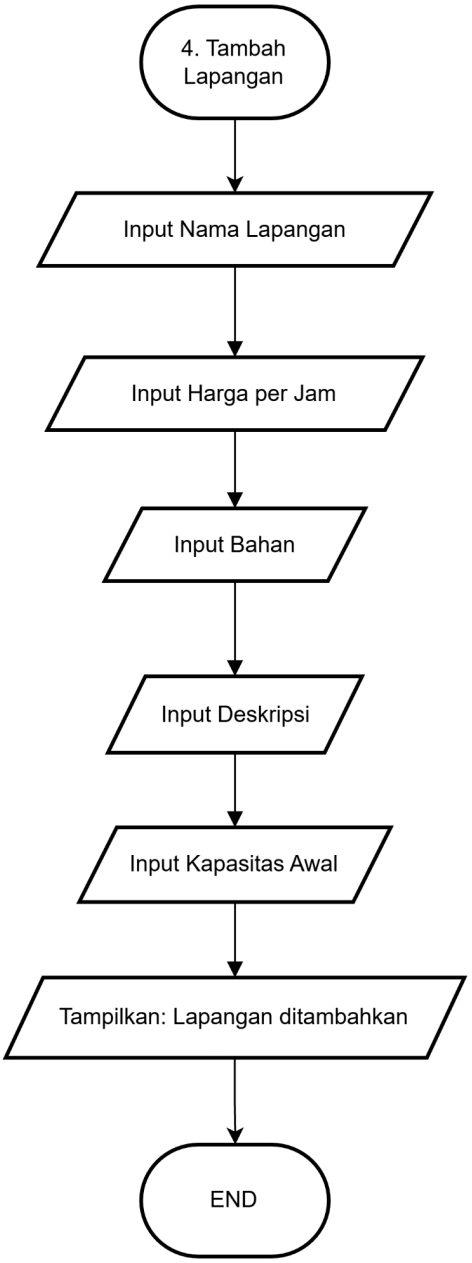

2. Registrasi
Proses Registrasi
Input Nama Pengguna Baru
Input Kata Sandi Baru
Tampilkan: Registrasi Berhasil!
END

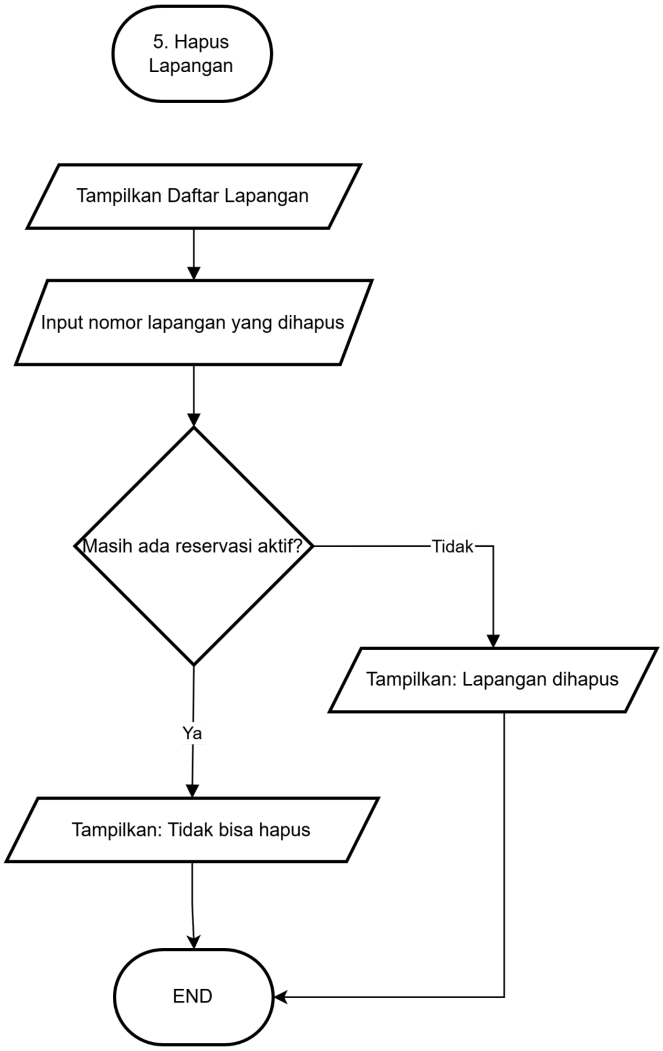

Menu Admin
Tampilkan Menu Admin
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
10. Cari Lapangan (Nama - Linear Search)
11. Cari Tagihan (Binary Search)
12. Keluar
Input Pilihan Menu
Pilihan = 1?
Pilihan = 2?
Pilihan = 3?
Pilihan = 4?
Pilihan = 5?
Pilihan = 6?
Pilihan = 7?
Pilihan = 8?
Pilihan = 9?
Pilihan = 10?
Pilihan = 11?
Pilihan = 12?
Tampilkan: Pilihan Tidak Valid
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
10. Cari Lapangan (Nama - Linear Search)
11. Cari Tagihan (Binary Search)
END

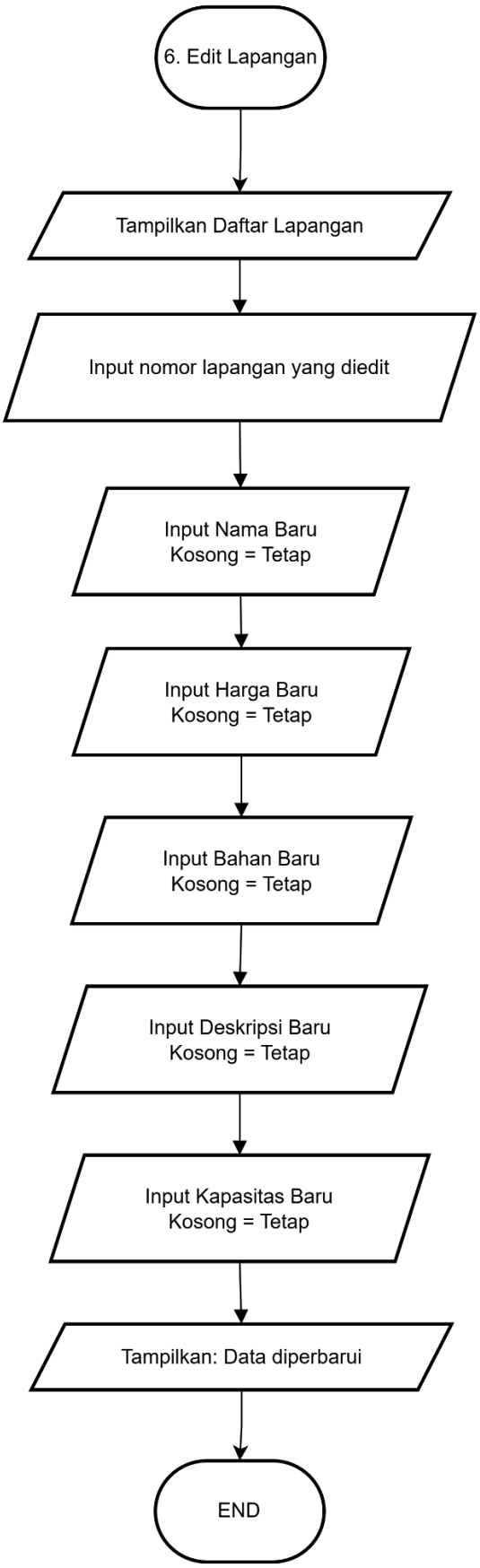

1. Status Lapangan
Tampilkan Status Lapangan
END

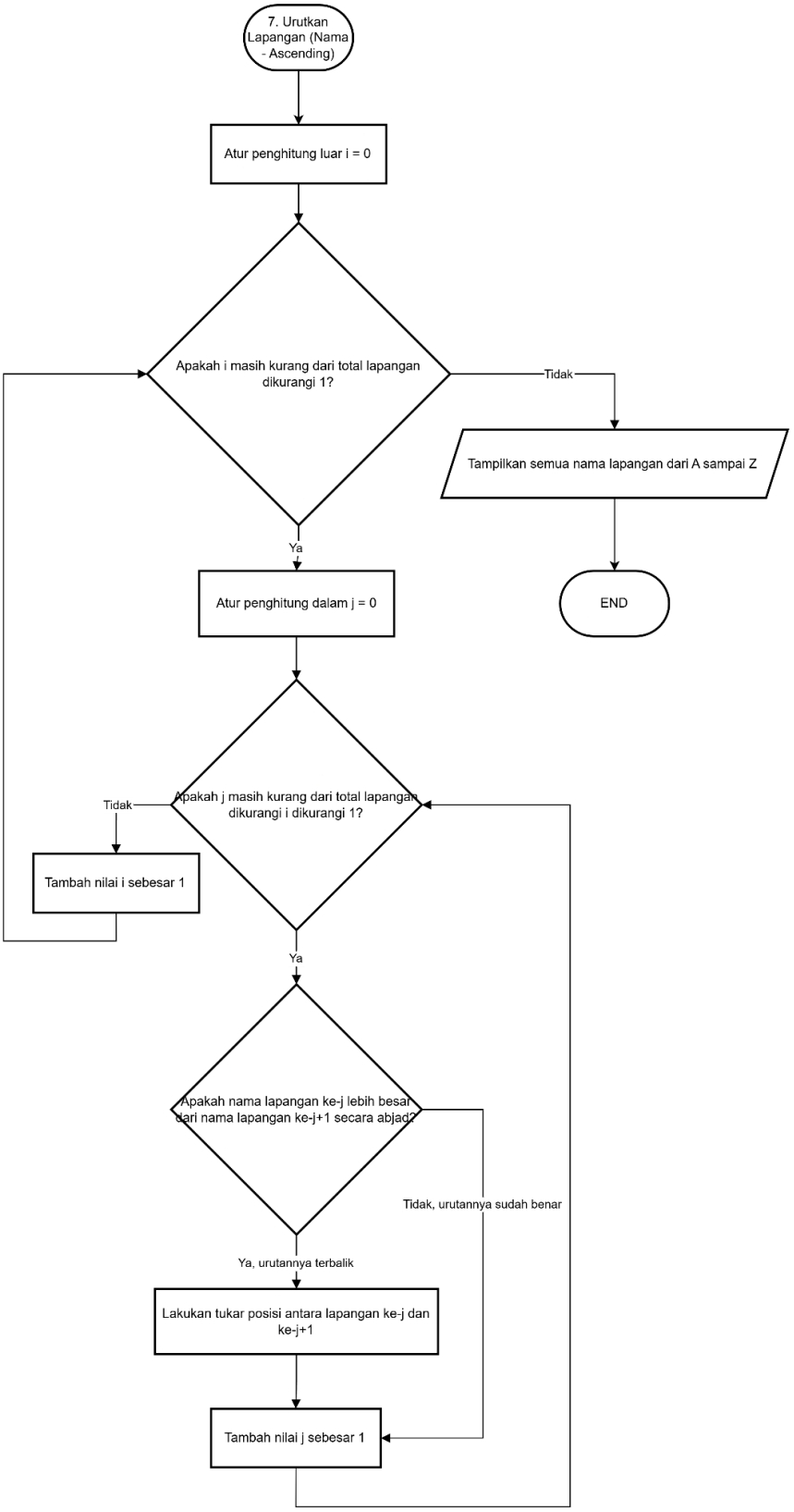

2. Lihat Rating
Tampilkan Daftar Rating
END

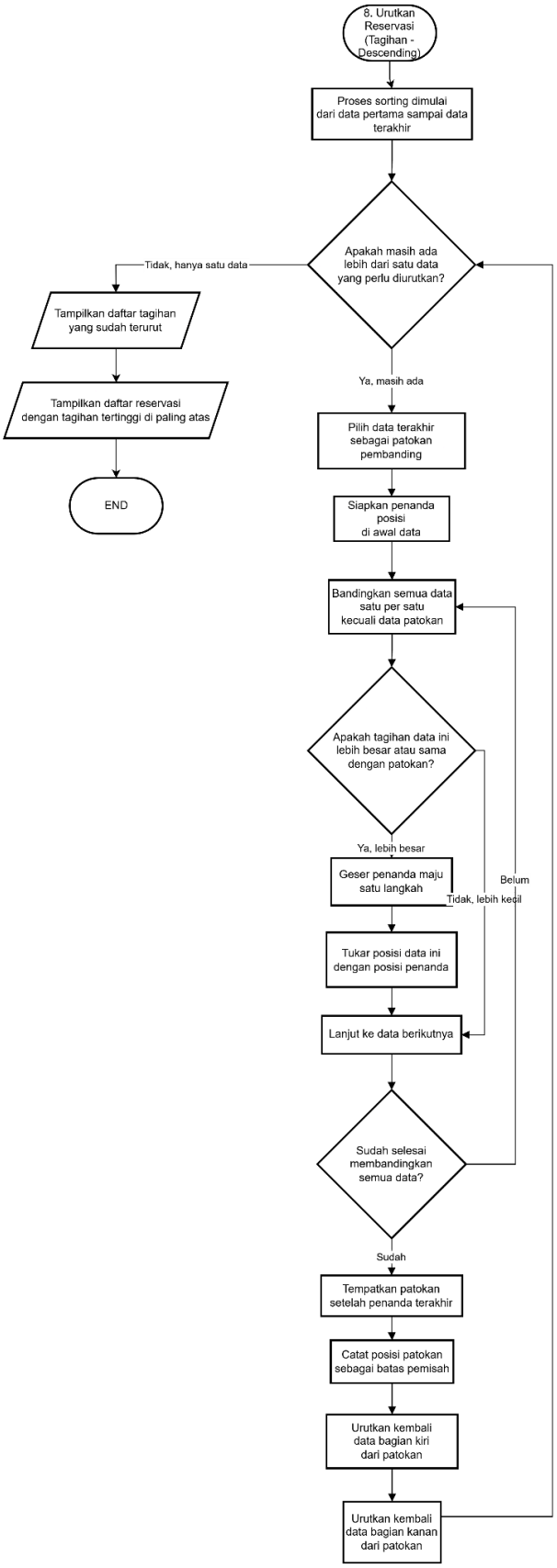

3. Hapus Ulasan
Tampilkan Daftar Rating
Input nomor rating yang dihapus
Tampilkan: Rating berhasil dihapus
END

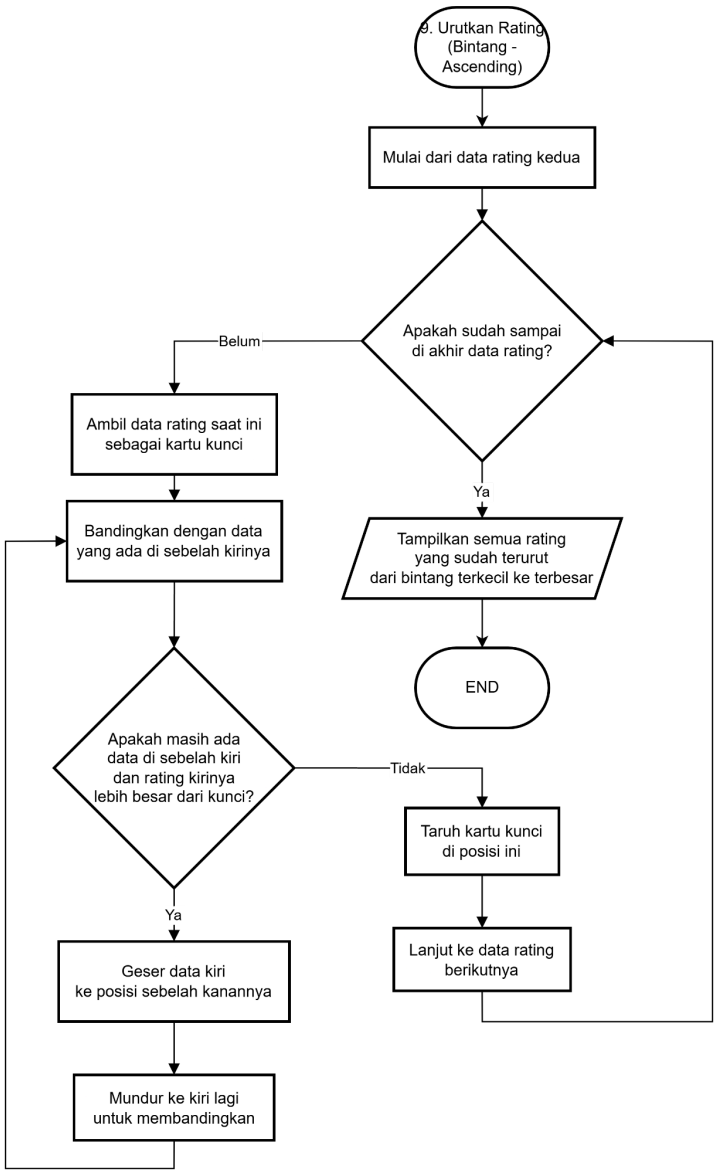

4. Tambah Lapangan
Input Nama Lapangan
Input Harga per Jam
Input Bahan
Input Deskripsi
Input Kapasitas Awal
Tampilkan: Lapangan ditambahkan
END

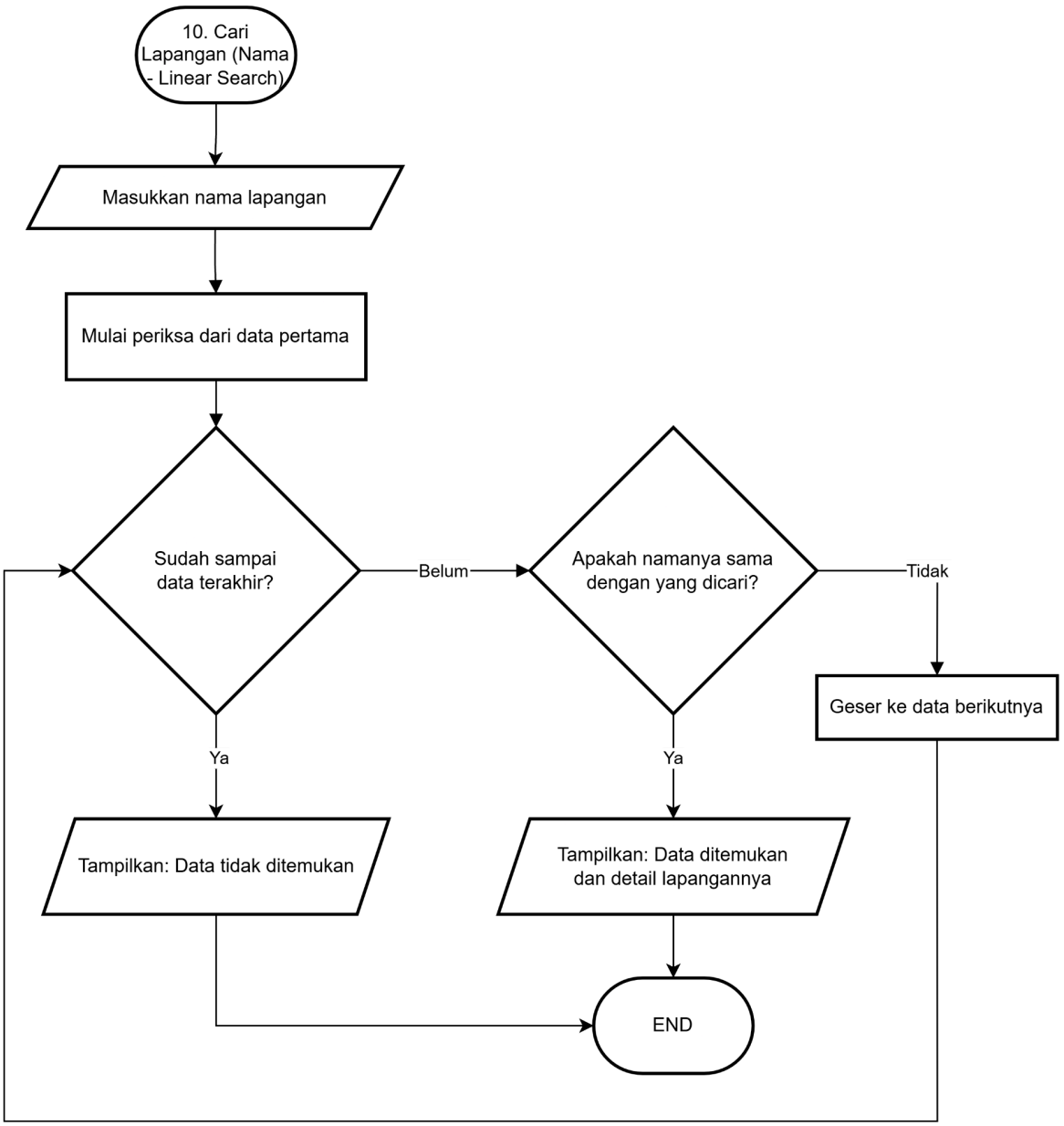

5. Hapus Lapangan
Tampilkan Daftar Lapangan
Input nomor lapangan yang dihapus
Masih ada reservasi aktif?
Tidak
Tampilkan: Lapangan dihapus
Ya
Tampilkan: Tidak bisa hapus
END

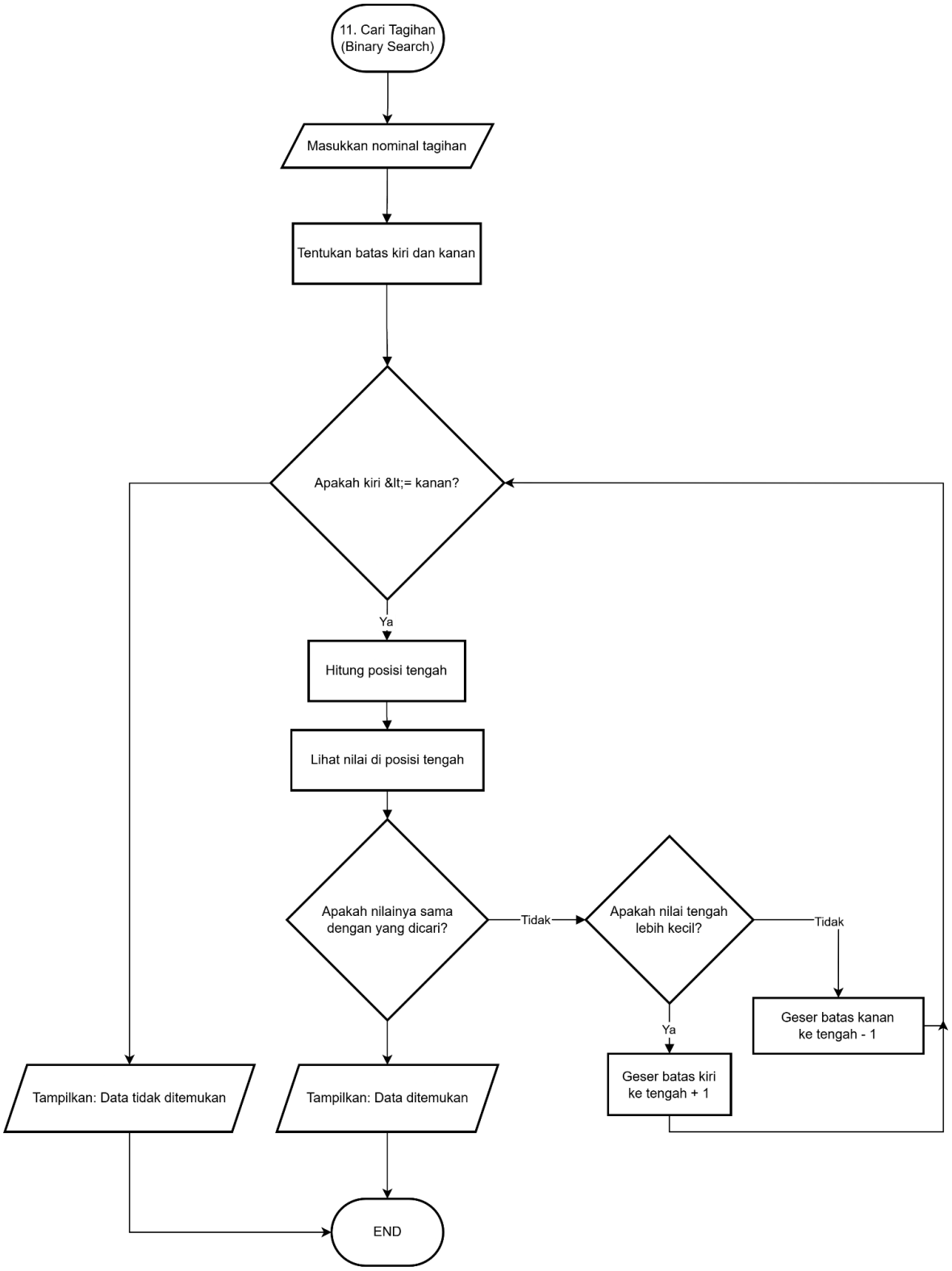

6. Edit Lapangan
Tampilkan Daftar Lapangan
Input nomor lapangan yang diedit
Input Nama Baru
Kosong = Tetap
Input Harga Baru
Kosong = Tetap
Input Bahan Baru
Kosong = Tetap
Input Deskripsi Baru
Kosong = Tetap
Input Kapasitas Baru
Kosong = Tetap
Tampilkan: Data diperbarui
END

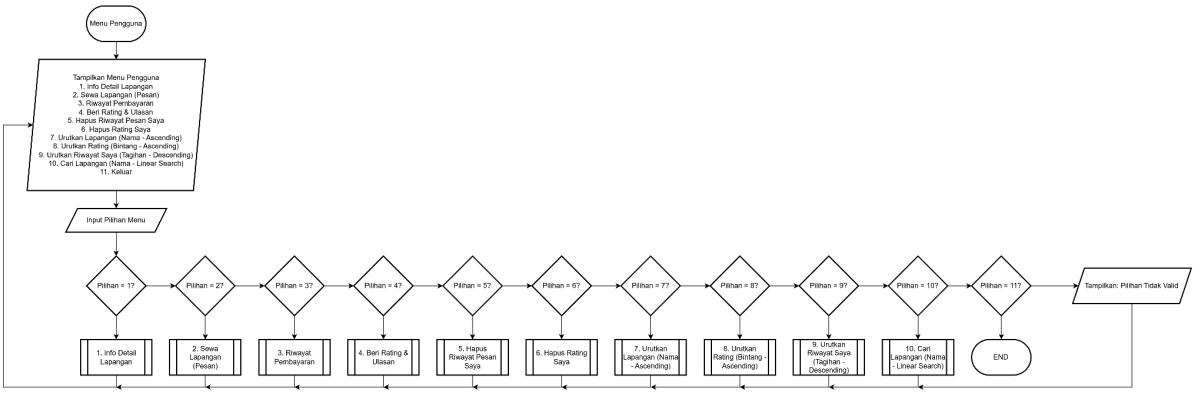

7. Urutkan Lapangan (Nama - Ascending)
Atur penghitung luar i = 0
Apakah i masih kurang dari total lapangan dikurangi 1?
Tidak
Tampilkan semua nama lapangan dari A sampai Z
END
Ya
Atur penghitung dalam j = 0
Apakah j masih kurang dari total lapangan dikurangi i dikurangi 1?
Tidak
Tambah nilai i sebesar 1
Ya
Apakah nama lapangan ke-j lebih besar dari nama lapangan ke-j+1 secara abjad?
Tidak, urutannya sudah benar
Ya, urutannya terbalik
Lakukan tukar posisi antara lapangan ke-j dan ke-j+1
Tambah nilai j sebesar 1

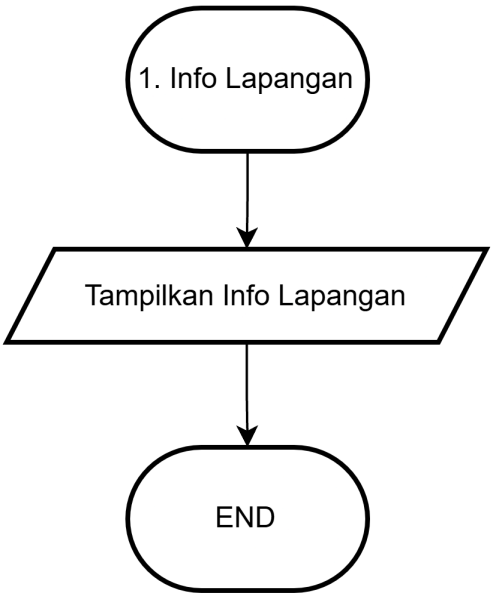

8. Urutkan Reservasi (Tagihan - Descending)
Proses sorting dimulai dari data pertama sampai data terakhir
Apakah masih ada lebih dari satu data yang perlu diurutkan?
Tidak, hanya satu data
Tampilkan daftar tagihan yang sudah terurut
Tampilkan daftar reservasi dengan tagihan tertinggi di paling atas
END
Ya, masih ada
Pilih data terakhir sebagai patokan pembanding
Siapkan penanda posisi di awal data
Bandingkan semua data satu per satu kecuali data patokan
Apakah tagihan data ini lebih besar atau sama dengan patokan?
Ya, lebih besar
Geser penanda maju satu langkah
Tidak, lebih kecil
Belum
Tukar posisi data ini dengan posisi penanda
Lanjut ke data berikutnya
Sudah selesai membandingkan semua data?
Sudah
Tempatkan patokan setelah penanda terakhir
Catat posisi patokan sebagai batas pemisah
Urutkan kembali data bagian kiri dari patokan
Urutkan kembali data bagian kanan dari patokan

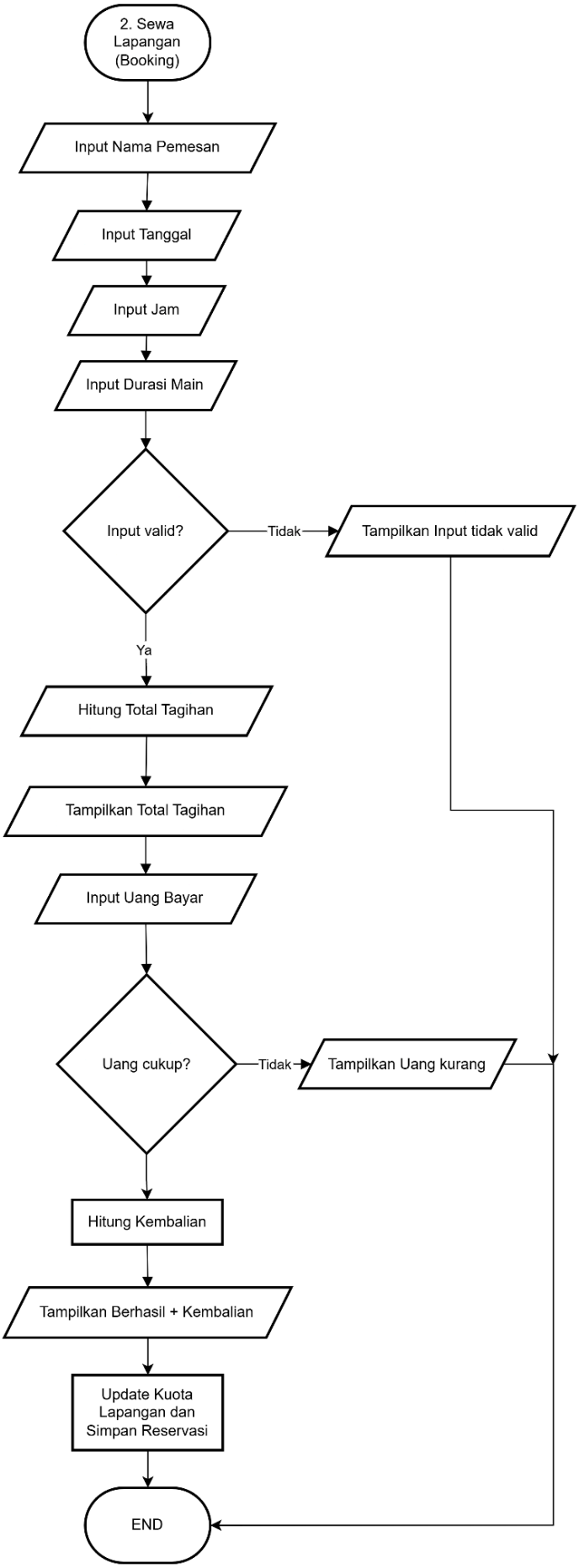

9. Urutkan Rating (Bintang - Ascending)
Mulai dari data rating kedua
Apakah sudah sampai di akhir data rating?
Belum
Ambil data rating saat ini sebagai kartu kunci
Bandingkan dengan data yang ada di sebelah kirinya
Apakah masih ada data di sebelah kiri dan rating kirinya lebih besar dari kunci?
Tidak
Taruh kartu kunci di posisi ini
Lanjut ke data rating berikutnya
Ya
Geser data kiri ke posisi sebelah kanannya
Mundur ke kiri lagi untuk membandingkan
Ya
Tampilkan semua rating yang sudah terurut dari bintang terkecil ke terbesar
END

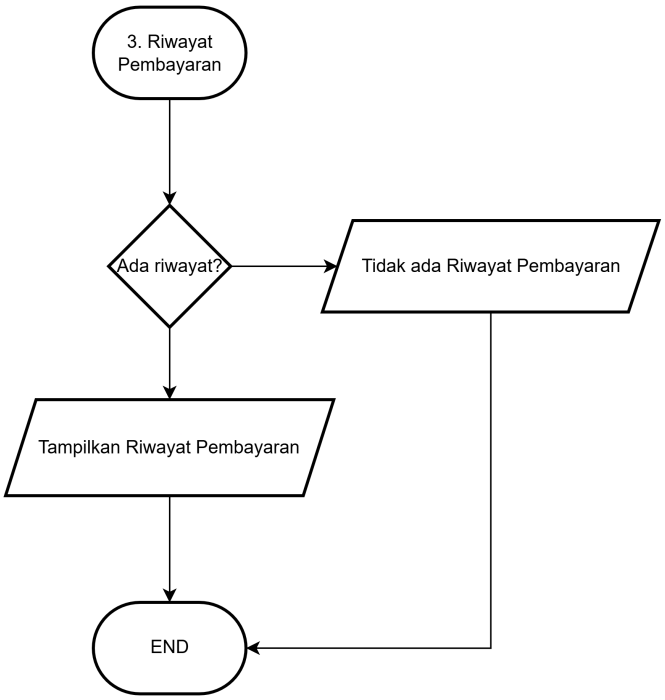

10. Cari Lapangan (Nama - Linear Search)
Masukkan nama lapangan
Mulai periksa dari data pertama
Sudah sampai data terakhir?
Belum
Apakah namanya sama dengan yang dicari?
Tidak
Geser ke data berikutnya
Ya
Ya
Tampilkan: Data tidak ditemukan
Tampilkan: Data ditemukan dan detail lapangannya
END

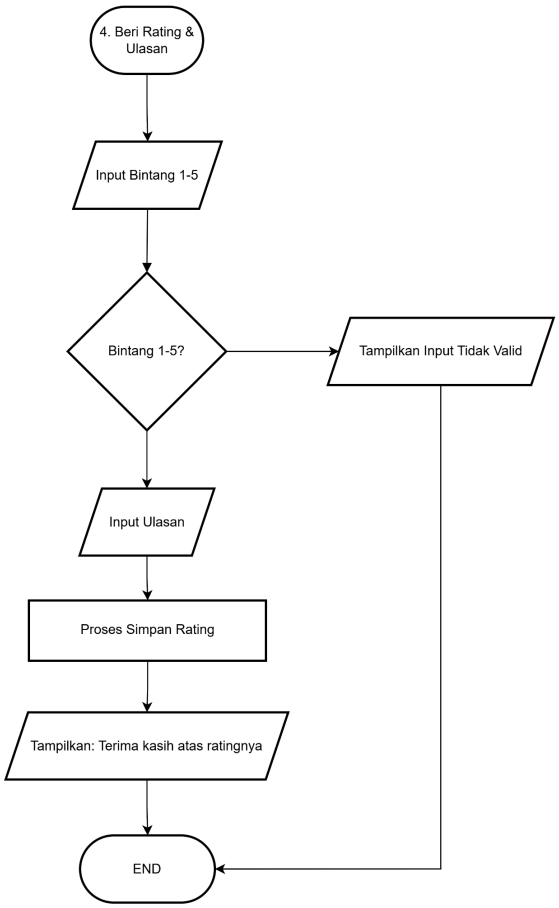

11. Cari Tagihan (Binary Search)
Masukkan nominal tagihan
Tentukan batas kiri dan kanan
Apakah kiri <= kanan?
Ya
Hitung posisi tengah
Lihat nilai di posisi tengah
Apakah nilainya sama dengan yang dicari?
Tidak
Apakah nilai tengah lebih kecil?
Tidak
Geser batas kanan ke tengah - 1
Ya
Geser batas kiri ke tengah + 1
Tampilkan: Data tidak ditemukan
Tampilkan: Data ditemukan
END

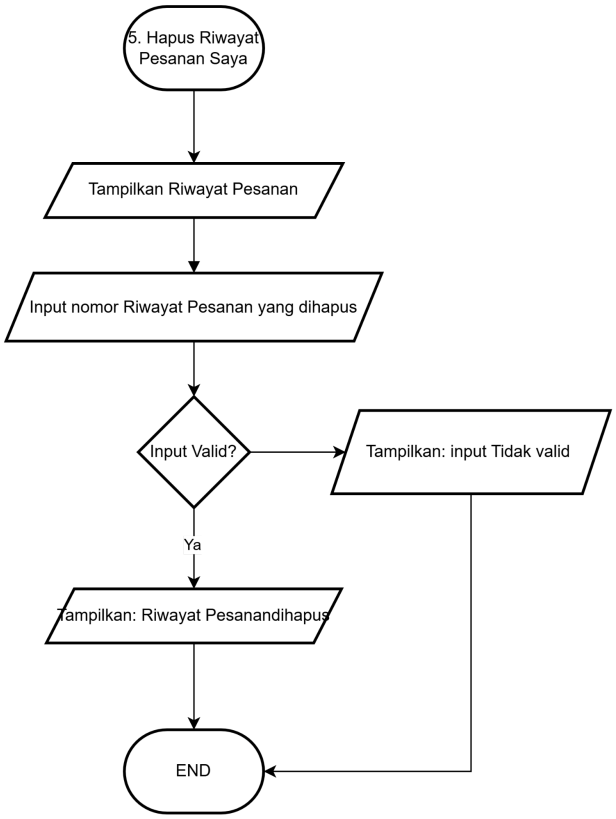

Menu Pengguna
Tampilkan Menu Pengguna
1. Info Detail Lapangan
2. Sewa Lapangan (Pesan)
3. Riwayat Pembayaran
4. Beri Rating & Ulasan
5. Hapus Riwayat Pesan Saya
6. Hapus Rating Saya
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Rating (Bintang - Ascending)
9. Urutkan Riwayat Saya (Tagihan - Descending)
10. Cari Lapangan (Nama - Linear Search)
11. Keluar
Input Pilihan Menu
Pilihan = 1?
Pilihan = 2?
Pilihan = 3?
Pilihan = 4?
Pilihan = 5?
Pilihan = 6?
Pilihan = 7?
Pilihan = 8?
Pilihan = 9?
Pilihan = 10?
Pilihan = 11?
Tampilkan: Pilihan Tidak Valid
1. Info Detail Lapangan
2. Sewa Lapangan (Pesan)
3. Riwayat Pembayaran
4. Beri Rating & Ulasan
5. Hapus Riwayat Pesan Saya
6. Hapus Rating Saya
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Rating (Bintang - Ascending)
9. Urutkan Riwayat Saya (Tagihan - Descending)
10. Cari Lapangan (Nama - Linear Search)
END

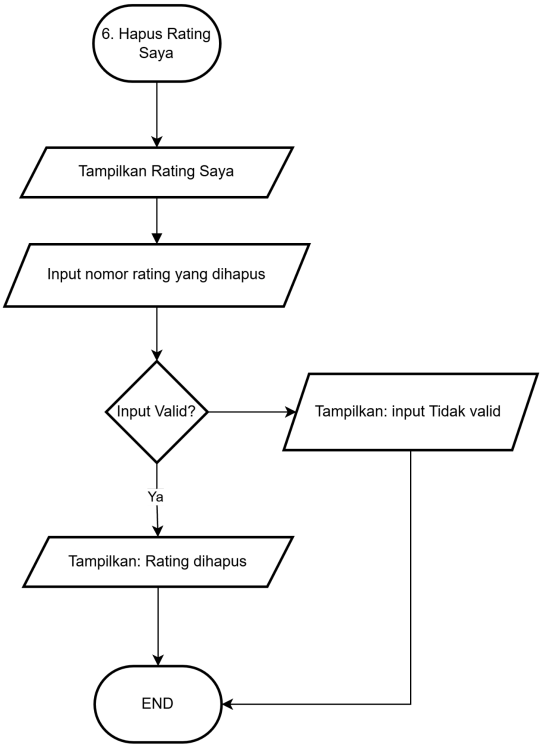

1. Info Lapangan
Tampilkan Info Lapangan
END

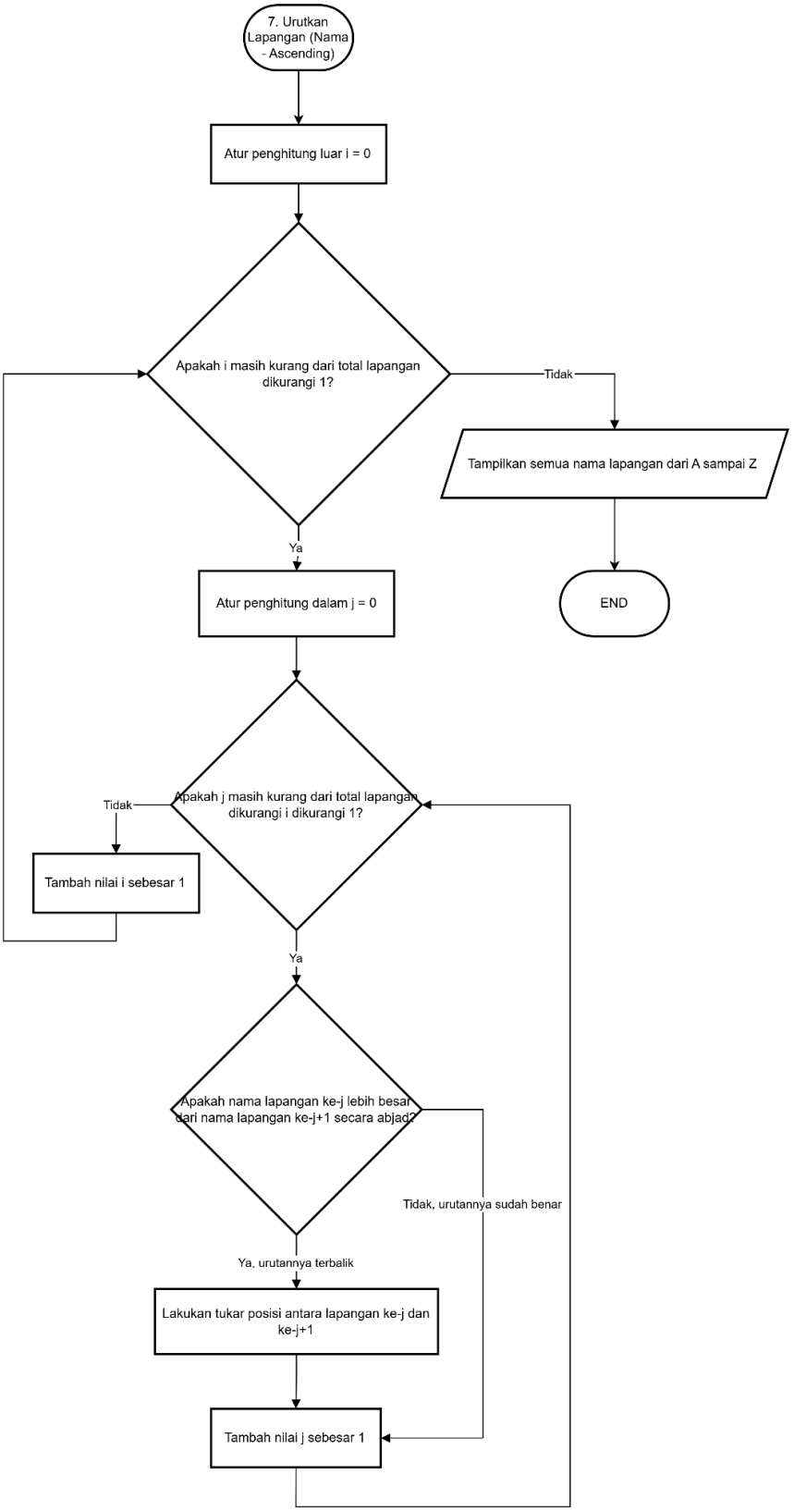

2. Sewa Lapangan (Booking)
Input Nama Pemesan
Input Tanggal
Input Jam
Input Durasi Main
Input valid?
Tidak
Tampilkan Input tidak valid
Ya
Hitung Total Tagihan
Tampilkan Total Tagihan
Input Uang Bayar
Uang cukup?
Tidak
Tampilkan Uang kurang
Hitung Kembalian
Tampilkan Berhasil + Kembalian
Update Kuota Lapangan dan Simpan Reservasi
END

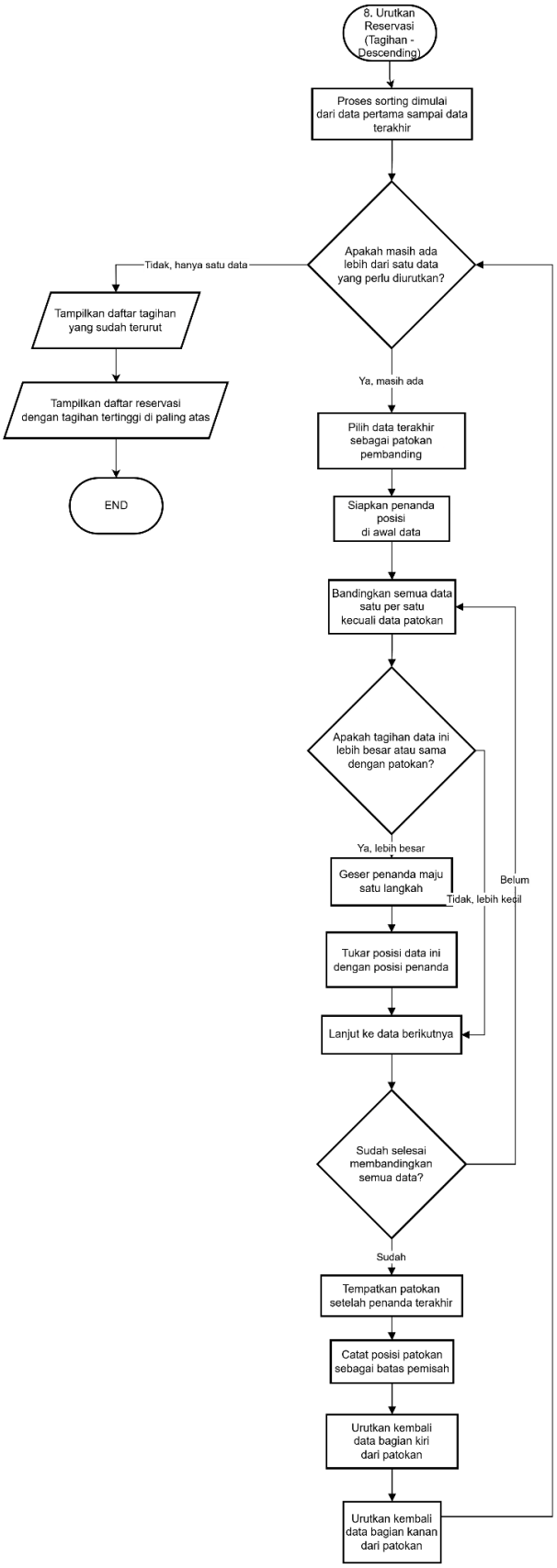

3. Riwayat Pembayaran
Ada riwayat?
Tidak ada Riwayat Pembayaran
Tampilkan Riwayat Pembayaran
END

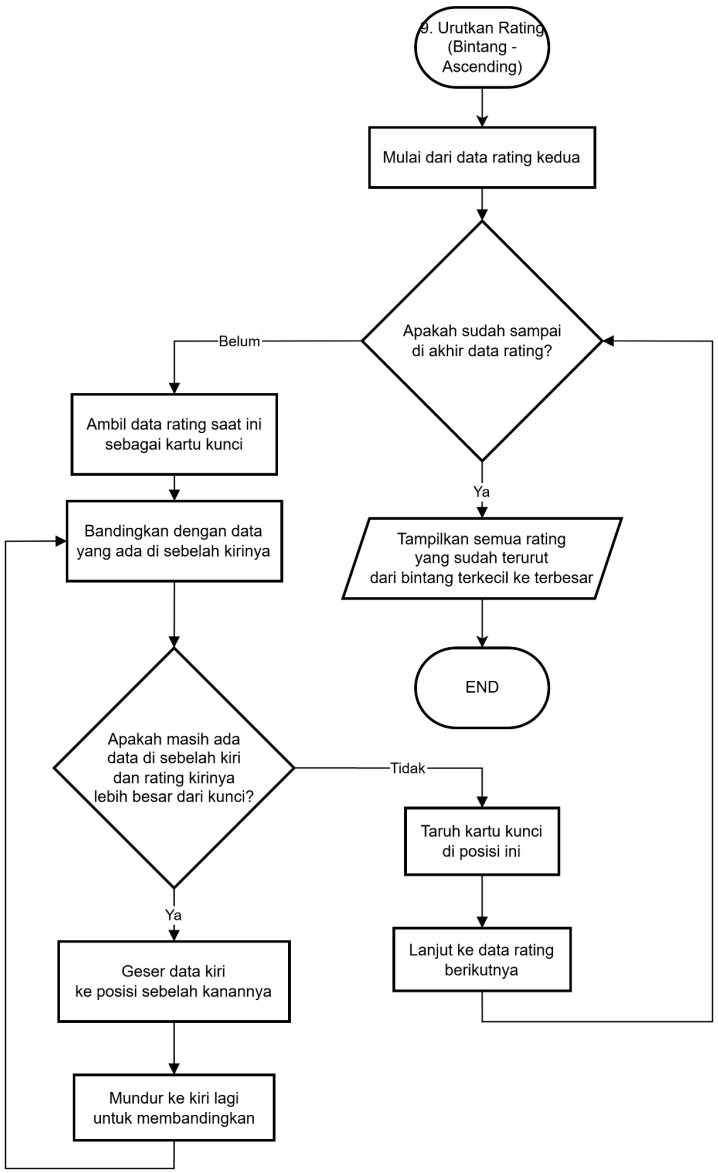

4. Beri Rating & Ulasan
Input Bintang 1-5
Bintang 1-5?
Tampilkan Input Tidak Valid
Input Ulasan
Proses Simpan Rating
Tampilkan: Terima kasih atas ratingnya
END

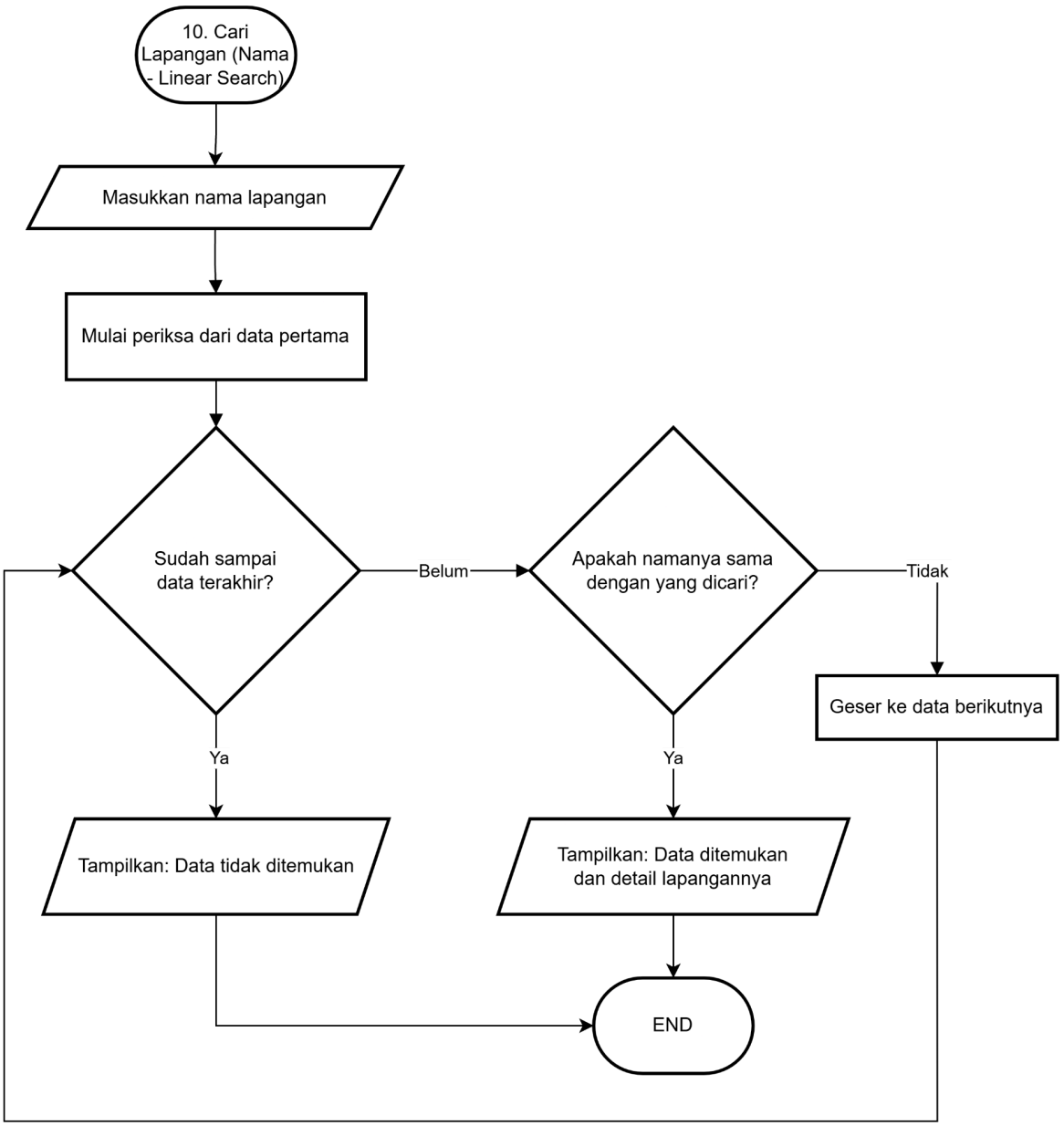

5. Hapus Riwayat Pesanan Saya
Tampilkan Riwayat Pesanan
Input nomor Riwayat Pesanan yang dihapus
Input Valid?
Tampilkan: input Tidak valid
Ya
Tampilkan: Riwayat Pesanandihapus
END


6. Hapus Rating Saya
Tampilkan Rating Saya
Input nomor rating yang dihapus
Input Valid?
Tampilkan: input Tidak valid
Ya
Tampilkan: Rating dihapus
END


7. Urutkan Lapangan (Nama - Ascending)
Atur penghitung luar i = 0
Apakah i masih kurang dari total lapangan dikurangi 1?
Tidak
Tampilkan semua nama lapangan dari A sampai Z
END
Ya
Atur penghitung dalam j = 0
Apakah j masih kurang dari total lapangan dikurangi i dikurangi 1?
Tidak
Tambah nilai i sebesar 1
Ya
Apakah nama lapangan ke-j lebih besar dari nama lapangan ke-j+1 secara abjad?
Tidak, urutannya sudah benar
Ya, urutannya terbalik
Lakukan tukar posisi antara lapangan ke-j dan ke-j+1
Tambah nilai j sebesar 1


8. Urutkan Reservasi (Tagihan - Descending)
Proses sorting dimulai dari data pertama sampai data terakhir
Apakah masih ada lebih dari satu data yang perlu diurutkan?
Tidak, hanya satu data
Tampilkan daftar tagihan yang sudah terurut
Tampilkan daftar reservasi dengan tagihan tertinggi di paling atas
END
Ya, masih ada
Pilih data terakhir sebagai patokan pembanding
Siapkan penanda posisi di awal data
Bandingkan semua data satu per satu kecuali data patokan
Apakah tagihan data ini lebih besar atau sama dengan patokan?
Ya, lebih besar
Geser penanda maju satu langkah
Tidak, lebih kecil
Belum
Tukar posisi data ini dengan posisi penanda
Lanjut ke data berikutnya
Sudah selesai membandingkan semua data?
Sudah
Tempatkan patokan setelah penanda terakhir
Catat posisi patokan sebagai batas pemisah
Urutkan kembali data bagian kiri dari patokan
Urutkan kembali data bagian kanan dari patokan


9. Urutkan Rating (Bintang - Ascending)
Mulai dari data rating kedua
Apakah sudah sampai di akhir data rating?
Belum
Ambil data rating saat ini sebagai kartu kunci
Bandingkan dengan data yang ada di sebelah kirinya
Apakah masih ada data di sebelah kiri dan rating kirinya lebih besar dari kunci?
Tidak
Taruh kartu kunci di posisi ini
Lanjut ke data rating berikutnya
Ya
Geser data kiri ke posisi sebelah kanannya
Mundur ke kiri lagi untuk membandingkan
Ya
Tampilkan semua rating yang sudah terurut dari bintang terkecil ke terbesar
END



1.1 Deskripsi Flowchart

Flowchart sistem GOR Badminton Sanjaya menggambarkan alur kerja program mulai dari proses awal dijalankan hingga program selesai. Sistem dirancang menggunakan struktur percabangan dan perulangan untuk mengatur jalannya menu dan proses transaksi.

2. Deskripsi Singkat Program

Sistem Reservasi GOR Badminton Sanjaya dibuat secara khusus untuk mengotomatisasi proses penyewaan lapangan bulu tangkis. Tujuan utamanya adalah memudahkan pelanggan dalam mengecek ketersediaan lapangan, melakukan penyewaan, dan memberikan ulasan. Di sisi lain, sistem ini memberikan kemudahan bagi pihak pengelola untuk memantau data inventaris lapangan dan masukan pelanggan secara langsung.

3. Source Code

3.1 User-Defined Library (GorLibrary.h)

Source Code:

```
#ifndef GOR_LIBRARY_H
#define GOR_LIBRARY_H

#include <iostream>
#include <stdexcept>
#include <string>

using namespace std;

class KurangBayarException : public runtime_error {
public:
    KurangBayarException(const string& msg, int kurang)
        : runtime_error(msg), jumlahKurang(kurang) {}

    int getJumlahKurang() const { return jumlahKurang; }

private:
    int jumlahKurang;
};

class DataTidakDitemukanException : public out_of_range {
public:
    DataTidakDitemukanException(const string& msg) : out_of_range(msg)
{}
};

inline void cetak_berwarna(const string& teks, int kode_warna) {
    cout << "\033[1;" << kode_warna << "m" << teks << "\033[0m";
```

```
}

inline void header_berwarna() {
    cetak_berwarna("\n=====================================\n", 36);
    cetak_berwarna("       GOR BADMINTON SANJAYA          \n", 36);
    cetak_berwarna("=====================================\n", 36);
}

#endif
```

Penjelasan:

Program ini menggunakan pustaka buatan pengguna bernama GorLibrary.h yang memuat kumpulan kelas exception kustom dan fungsi utilitas. Library ini dirancang khusus untuk memenuhi kebutuhan sistem manajemen GOR Badminton.

3.2 Error Handling dengan Exception Handling (Try, Throw, Catch)

Source Code:

```
void proses_pesan(string nama_pengguna, Lapangan* daftar_lapangan,
                  int jumlah_lapangan, Reservasi* daftar_reservasi,
                  int& jumlah_reservasi) {
    try {
        if (jumlah_reservasi >= MAX_DATA) {
            cetak_berwarna("[!] Kapasitas sistem reservasi penuh.\n",
31);
            return;
        }

        cout << "\n--- PILIH LAPANGAN ---" << endl;
        for(int indeks=0; indeks<jumlah_lapangan; indeks++)
            cout << indeks+1 << ". " << (*(daftar_lapangan +
indeks)).nama
                << " (Tersedia: " << (*(daftar_lapangan +
indeks)).tersedia << ")" << endl;

        cout << jumlah_lapangan+1 << ". Batal & Kembali" << endl;
        cout << "Pilih (1-" << jumlah_lapangan+1 << "): ";
        int pilih_lapangan = masukan_angka();

        if (pilih_lapangan == jumlah_lapangan+1) return;
        if (pilih_lapangan < 1 || pilih_lapangan > jumlah_lapangan)
            throw logic_error("Nomor lapangan tidak valid!");

        int indeks = pilih_lapangan - 1;
        if ((*(daftar_lapangan + indeks)).tersedia > 0) {
            cout << "\n--- FORM PEMESANAN [" << (*(daftar_lapangan +
```

```
indeks)).nama << "] ---" << endl;
            bersihkan_masukan();
            cout << "Nama Pemesan      : ";
            getline(cin, (*(daftar_reservasi +
jumlah_reservasi)).namaPemesan);
            cout << "Tanggal : ";
            cin >> (*(daftar_reservasi +
jumlah_reservasi)).jadwal.tanggal;
            cout << "Jam     : ";
            cin >> (*(daftar_reservasi + jumlah_reservasi)).jadwal.jam;
            cout << "Durasi  : ";
            (*(daftar_reservasi + jumlah_reservasi)).jadwal.durasi =
masukan_angka();

            if ((*(daftar_reservasi + jumlah_reservasi)).jadwal.durasi
<= 0)
                throw logic_error("Durasi main tidak boleh 0 atau
negatif!");

            int total = (*(daftar_lapangan + indeks)).harga *
(*(daftar_reservasi + jumlah_reservasi)).jadwal.durasi;
            cout << "\nTOTAL TAGIHAN : Rp" << total << "\nMasukkan Uang
: Rp";
            int bayar = masukan_angka();

            if (bayar < total)
                throw KurangBayarException("Uang pembayaran kurang!",
total - bayar);

            // Finalisasi Data Reservasi
            (*(daftar_reservasi + jumlah_reservasi)).totalTagihan =
total;
            (*(daftar_reservasi + jumlah_reservasi)).uangBayar = bayar;
            (*(daftar_reservasi + jumlah_reservasi)).kembalian = bayar -
total;
            (*(daftar_reservasi + jumlah_reservasi)).namaLapangan =
(*(daftar_lapangan + indeks)).nama;
            (*(daftar_reservasi + jumlah_reservasi)).usernamePemilik =
nama_pengguna;
            (*(daftar_lapangan + indeks)).tersedia--;

            if (bayar == total)
                tampilkan_pesan_string("BERHASIL! (Uang Pas)");
            else
                tampilkan_pesan_nilai("BERHASIL! Kembalian",
bayar-total);
```

```
            jumlah_reservasi++;
        } else {
            throw runtime_error("Lapangan sedang penuh!");
        }
    } catch (const KurangBayarException& e) {
        cetak_berwarna("\n[!] TRANSAKSI DITOLAK: " + string(e.what()),
31);
        cout << "[!] Anda kurang membayar sebesar: Rp" <<
e.getJumlahKurang() << endl;
    } catch (const logic_error& e) {
        cetak_berwarna("\n[!] KESALAHAN LOGIKA: " + string(e.what()),
31);
    } catch (const runtime_error& e) {
        cetak_berwarna("\n[!] KESALAHAN SISTEM: " + string(e.what()),
31);
    }
}
```

Penjelasan:
Program ini menerapkan mekanisme Exception Handling untuk memastikan validitas data dan stabilitas aplikasi melalui tiga blok utama:

1. Block Try: Membungkus seluruh logika pemesanan yang berpotensi menimbulkan kesalahan (input pengguna, ketersediaan lapangan, dan transaksi pembayaran).
2. Block Throw: Melemparkan pengecualian saat kondisi tidak terpenuhi, seperti input nomor lapangan yang salah, durasi negatif, atau kekurangan uang pembayaran.
3. Block Catch: Menangkap pengecualian yang dilempar untuk kemudian diproses tanpa menghentikan paksa (crash) program. Hal ini memungkinkan pengguna mendapatkan pesan kesalahan yang informatif (misalnya, jumlah nominal uang yang kurang).

3.3 Penggunaan Library Eksternal dan Internal

Source Code:

```
#include "GorStruk.h"      // Library kustom (header utama)
#include "GorLibrary.h"    // Library kustom (exception & utility)
#include <iostream>        // Library internal (Input/Output)
#include <iomanip>         // Library internal (Formatting)
#include <limits>          // Library internal (Numeric limits)
#include <string>          // Library internal (String manipulation)
#include <stdexcept>       // Library internal (Standard exceptions)
#include <ctime>           // Library internal (Time functions)
#include <cmath>           // Library internal (Math functions)

int main() {
    // Inisialisasi Data
    User daftar_pengguna[MAX_DATA];
    Reservasi daftar_reservasi[MAX_DATA];
```

```
    Rating daftar_rating[MAX_DATA];
    Lapangan daftar_lapangan[MAX_DATA];

    int jumlah_pengguna = 0, jumlah_reservasi = 0,
        jumlah_rating = 0, jumlah_lapangan = 0;

    // Konfigurasi Lapangan Awal
    daftar_lapangan[0] = {"Vinyl Premium", 60000, "Rubber Vinyl",
"Standar BWF", 1, 1};
    daftar_lapangan[1] = {"Karpet Standar", 40000, "Sintetis", "Latihan
Pro", 2, 2};
    daftar_lapangan[2] = {"Semen Beton", 25000, "Semen Halus",
"Ekonomis", 2, 2};

    jumlah_lapangan = 3;
    daftar_pengguna[0] = {"firman", "000"};
    jumlah_pengguna = 1;

    string nama_pengguna_masuk;
    int pilihan_utama;

    while (true) {
        try {
            tampilkan_header(daftar_rating, jumlah_rating);
            cetak_berwarna("1. Masuk\n2. Registrasi\n3. Keluar\nPilih
menu: ", 34);
            pilihan_utama = masukan_angka();

            if (pilihan_utama == 1) {
                if (proses_masuk(daftar_pengguna, jumlah_pengguna,
nama_pengguna_masuk)) {
                    if (nama_pengguna_masuk == "adminGor")
                        menu_admin(daftar_lapangan, jumlah_lapangan,
daftar_rating,
                                   jumlah_rating, daftar_reservasi,
jumlah_reservasi);
                    else
                        menu_pengguna(nama_pengguna_masuk,
daftar_lapangan,
                                      jumlah_lapangan, daftar_reservasi,
daftar_rating,
                                      jumlah_reservasi, jumlah_rating);
                } else {
                    throw logic_error("Batas percobaan login habis.");
                }
            }
```

```
            else if (pilihan_utama == 2)
                proses_registrasi(daftar_pengguna, jumlah_pengguna);
            else if (pilihan_utama == 3) {
                cetak_berwarna("\nTerima kasih!\n", 32);
                break;
            }
            else {
                throw invalid_argument("Nomor menu tidak valid.");
            }
        } catch (const logic_error& e) {
            cetak_berwarna("\n[!] PERINGATAN: " + string(e.what()) +
"\n", 33);
        } catch (const exception& e) {
            cetak_berwarna("\n[!] SYSTEM ERROR: " + string(e.what()) +
"\n", 31);
        }
    }
    return 0;
}
```

Penjelasan:

Dalam pengembangan program GOR Badminton Sanjaya, digunakan kombinasi antara Library Internal (Standard Library C++) dan Library Eksternal (User-Defined Library). Pendekatan ini bertujuan untuk menciptakan sistem yang stabil, modular, dan memiliki antarmuka yang interaktif.

1. Library Internal (Standard C++ Library)

Library ini adalah pustaka bawaan C++ yang digunakan untuk menangani fungsi-fungsi fundamental program:

- <iostream>: Digunakan untuk keperluan standar input dan output data.
- <iomanip>: Digunakan untuk memformat tampilan output agar rapi, seperti pengaturan lebar kolom tabel laporan.
- <string>: Digunakan untuk memproses dan memanipulasi data teks secara dinamis.
- <stdexcept>: Menyediakan kerangka dasar penanganan kesalahan (standard exceptions).
- <limits>: Digunakan untuk membersihkan input buffer, mencegah terjadinya error saat pengguna memasukkan data yang tidak valid.
- <ctime> & <cmath>: Digunakan untuk pengolahan data waktu operasional dan perhitungan matematis biaya sewa.

2. Library Eksternal (User-Defined Library)

Library eksternal dalam program ini adalah file header yang dibuat secara mandiri untuk mengelompokkan logika program agar lebih terorganisir:

- "GorStruk.h": Berfungsi sebagai pusat penyimpanan struktur data (struct) seperti data Lapangan, Reservasi, dan User, serta memuat prototipe fungsi utama.
- "GorLibrary.h": Pustaka khusus yang menangani kelas exception kustom (misalnya penanganan kurang bayar) dan fungsi utilitas visual untuk menampilkan teks berwarna di terminal.

3. Mekanisme Integrasi

Program membedakan cara pemanggilan kedua jenis library ini sebagai instruksi bagi compiler:

- Tanda Kurung Siku < >: Memberitahu compiler untuk mencari library di dalam direktori sistem C++ (Library Internal).

- Tanda Kutip Dua " ": Memberitahu compiler untuk mencari library di dalam folder proyek lokal (Library Eksternal).

4. Hasil Output

```
--- PENCARIAN LAPANGAN (LINEAR SEARCH) ---
Masukkan nama lapangan yang dicari: Semen Beton

>>> LAPANGAN DITEMUKAN! <<<
Nama        : Semen Beton
Harga/Jam   : Rp25000
Bahan       : Semen Halus
Deskripsi   : Ekonomis
Tersedia    : 2 dari 2
```

Gambar 4.1:Admin dan user dapat melakukan pencarian lapangan karena ini tersedia di menu admin dan pengguna.

```
--- PENCARIAN TAGIHAN (BINARY SEARCH) ---
Catatan: Data harus diurutkan terlebih dahulu (Opsi 8)
Masukkan nominal tagihan yang dicari: Rp130000

[!] Tagihan Rp130000 tidak ditemukan.
```

Gambar 4.2:admin mencoba mencari tagihan spesifik, di karena tidak ditemukan, maka diberi opsi untuk memilih opsi 8 untuk mengurutkan.

```
--- PENCARIAN TAGIHAN (BINARY SEARCH) ---
Catatan: Data harus diurutkan terlebih dahulu (Opsi 8)
Masukkan nominal tagihan yang dicari: Rp120000

>>> TAGIHAN DITEMUKAN! <<<
Pemesan      : Denny
Lapangan     : Vinyl Premium
Tanggal      : 07-03
Jam          : 20:00
Total        : Rp120000
Bayar        : Rp150000
Kembalian    : Rp30000
```

```
--- PENCARIAN TAGIHAN (BINARY SEARCH) ---
Catatan: Data harus diurutkan terlebih dahulu (Opsi 8)
Masukkan nominal tagihan yang dicari: Rp120000

>>> TAGIHAN DITEMUKAN! <<<
Pemesan      : Denny
Lapangan     : Vinyl Premium
Tanggal      : 07-03
Jam          : 20:00
Total        : Rp120000
Bayar        : Rp150000
Kembalian    : Rp30000
```

Gambar 4.3:admin sudah mengurutkan dan mencoba cari nominal spesifik dan berhasil menemukan.

5. Langkah-langkah GIT

5.1 GIT Add

```
>git add .
```

git add .

Perintah git add digunakan untuk memasukkan perubahan file ke dalam staging area.

Staging area adalah tempat sementara yang digunakan untuk menyiapkan file sebelum dilakukan commit dan tanda titik "." berarti menambahkan semua file yang ada dalam folder proyek.

5.2 GIT Commit

```
D:\Praktikum Semester 2\APL\Praktikum-APL\POST-TEST\post-test-apl-7>git commit -m "posttest-7"
[main 5ac1c63] posttest-7
 5 files changed, 712 insertions(+)
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-7/GorApp.exe
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-7/GorFungsi.cpp
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-7/GorLibrary.h
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-7/GorSistem.cpp
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-7/GorStruk.h
```

git commit -m “posttest-7”

GIT Commit adalah proses menyimpan snapshot perubahan file ke dalam repository Git.
Setiap commit akan memiliki ID unik (hash), waktu commit, penulis commit, dan pesan commit.
Pesan commit (-m) digunakan untuk menjelaskan perubahan yang dilakukan.

5.3 GIT Push

```
D:\Praktikum Semester 2\APL\Praktikum-APL\POST-TEST\post-test-apl-7>git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 79.27 KiB | 6.61 MiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/AxyL1a/Praktikum_APL.git
   2f310d0..5ac1c63  main -> main

D:\Praktikum Semester 2\APL\Praktikum-APL\POST-TEST\post-test-apl-7>
```

git push origin main
Perintah ini digunakan untuk mengirim seluruh commit dari repository lokal ke repository GitHub.